

DIGITAL STACK

Knowledge. Performance. Growth.

SESIUNEA 4: AGENȚI ÎN PYTHON

AGENDĂ

- Ce este un Agent AI?
- Arhitectura unei aplicații bazate pe agenți
- Conversation Context
- System Prompts
- Tool Calling
- Model Context Protocol (MCP)
- LangGraph
- Hands-on: Construirea unui Agent AI local folosind Ollama

Ce este un Agent AI?

Un Agent AI este o aplicație care folosește un model de limbaj pentru a rezolva sarcini și a interacționa cu alte componente software.

Spre deosebire de un chatbot simplu, un agent poate:

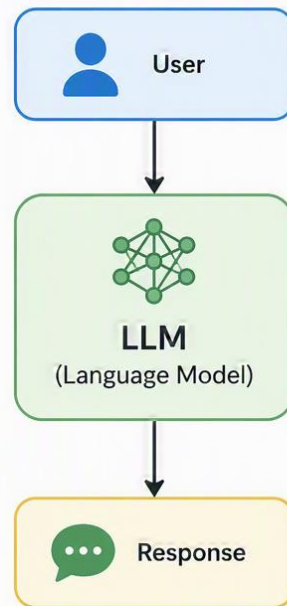
- păstra contextul conversației;
- utiliza tool-uri externe;
- lua decizii;
- executa acțiuni

Un agent este mai mult decât un model. Este un sistem construit în jurul modelului..

LLM vs Agent

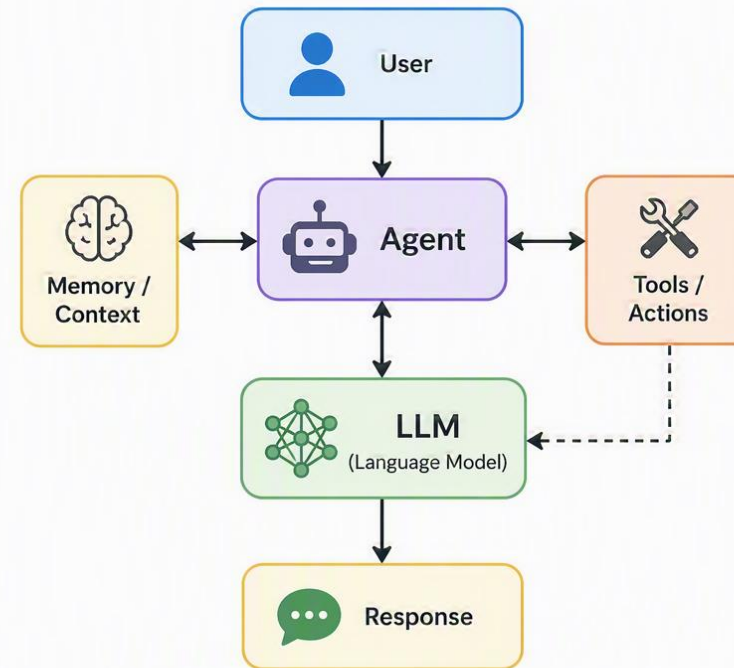
LLM

Generates text based on the input without external interaction



Agent

Uses a model, memory and tools to take actions and solve tasks

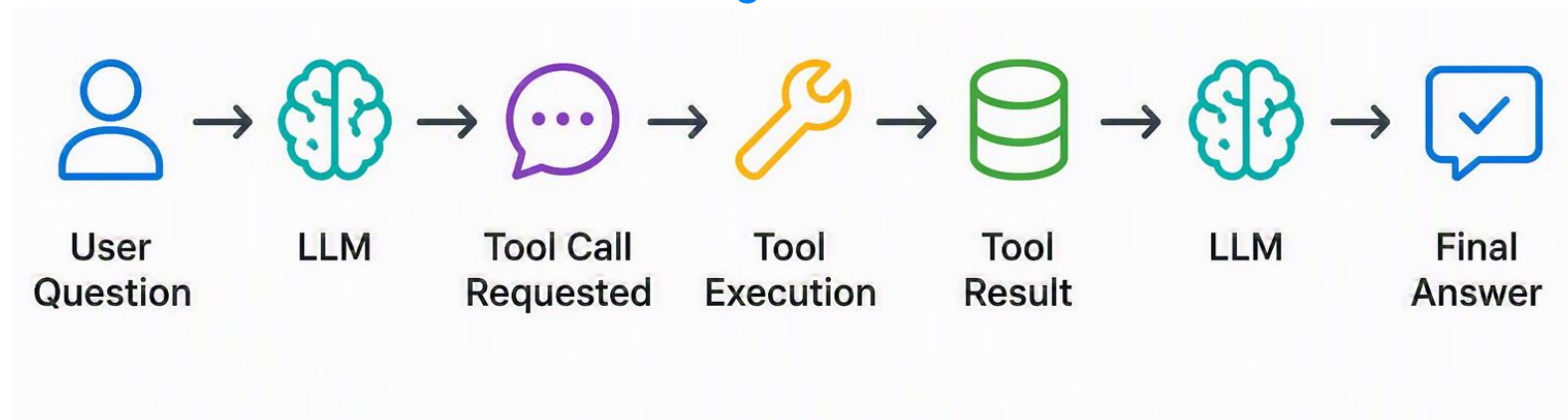


Conversation Context, System Prompts & Tool Calling

Conversation Context

- istoricul conversației;
- oferă memorie agentului;
- trimis la fiecare request.

Tool Calling



System Prompt

- definește personalitatea agentului;
- stabilește reguli și restricții;
- influențează toate răspunsurile.

Specification Driven Development

Înainte de implementare definim:

- responsabilitățile;
- interfețele;
- contractele;
- fluxurile aplicației.

Avantaje

- cod mai clar;
- extensibilitate;
- testare mai simplă.

Framework Architecture

Separăm aplicația în componente independente.

Avantaje:

- cod mai organizat;
- mentenanță mai ușoară;
- reutilizare mai bună.

Caracteristicile unui cod înainte de review

- Correctness
- Robustness
- Scalability
- Clarity

Ce este MCP?

MCP (Model Context Protocol) este un standard pentru integrarea tool-urilor cu modele AI.

Permite:

- descrierea tool-urilor;
- descoperirea tool-urilor;
- apelarea tool-urilor;
- schimbul standardizat de informații.

MCP standardizează modul în care agenții comunică cu tool-urile.

Ce este LangGraph?

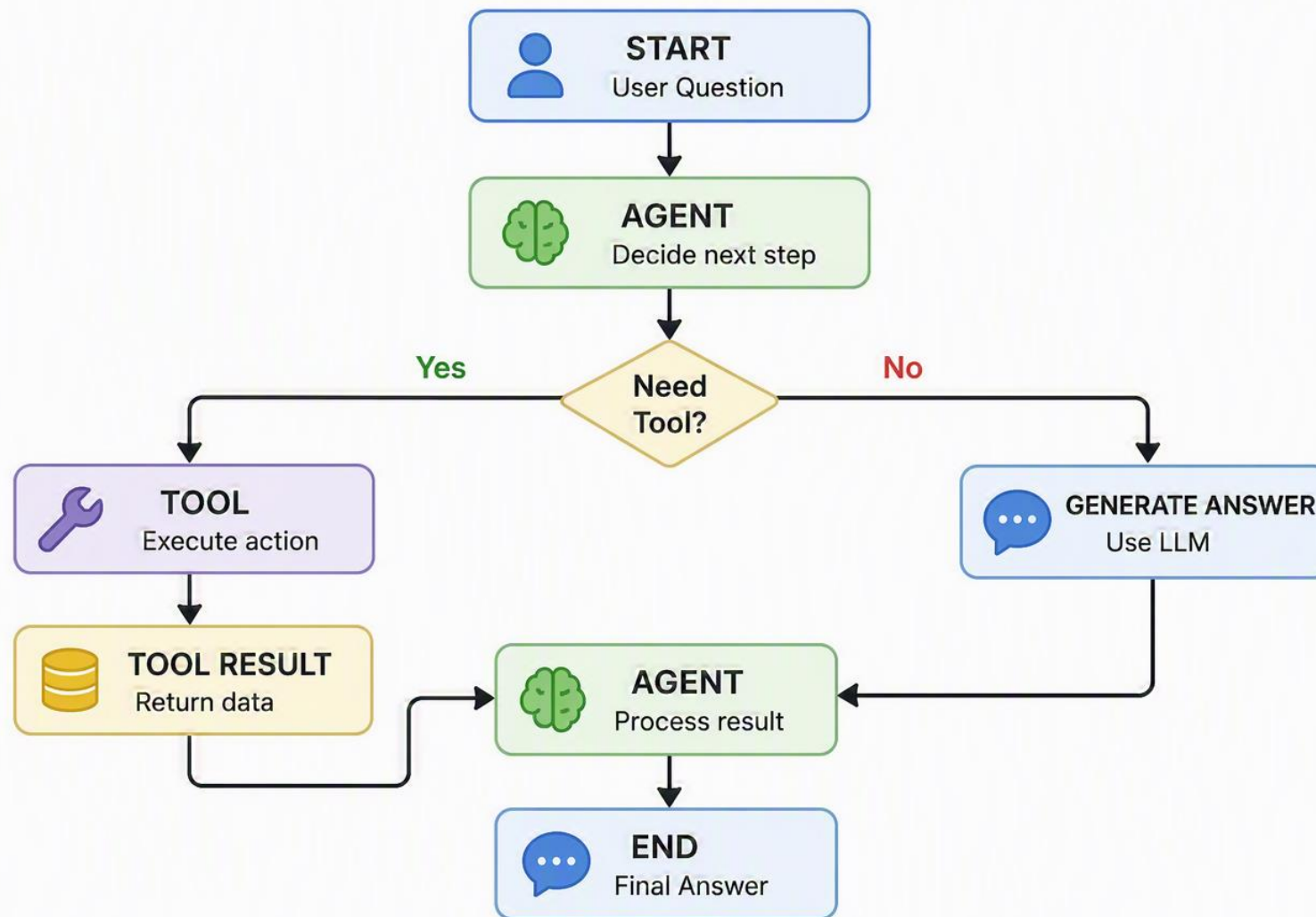
LangGraph este un framework pentru construirea agenților AI.

Modelează fluxul aplicației ca un graf.

Concepte importante:

- Node
- Edge
- State

LangGraph: Example



Ollama Cheat Sheet

Instalare

- **Windows**

1. Descărcați installer-ul: <https://ollama.com/download>
2. Rulați installer-ul și urmați pașii din wizard.

- **Linux**

```
>>> curl -fsSL https://ollama.com/install.sh | sh
```

Verificare instalare

```
>>> ollama --version
```

Descărcare model

```
>>> ollama pull qwen3:1.7b
```

Vizualizare modele instalate

```
>>> ollama list
```

Ștergere model

```
>>> ollama rm qwen3:1.7b
```

Pornire server Ollama

Ollama pornește automat serviciul local. Endpoint implicit:

`http://localhost:11434`

Test API

```
>>> curl http://localhost:11434/api/generate -d
'{"model":"qwen3:1.7b","prompt":"Say
hello","stream":false}'
```

Recommended Models

Model	Dimensiune	RAM recomandat	Viteză	Calitate	Recomandare
qwen3:0.6b	~0.5 GB	4+ GB	Foarte rapid	Basic	Ideal pentru laborator
qwen3:1.7b	~1.4 GB	8+ GB	Rapid	Bun	Recomandat pentru majoritatea participanților
qwen3:4b	~2.5-3 GB	8-16 GB	Mediu	Foarte bun	Pentru laptopuri mai puternice
qwen3:8b	~5 GB	16+ GB	Mai lent	Excelent	Nu este recomandat pentru workshop
llama3.2:3b	~2 GB	8+ GB	Rapid	Bun	Alternativă foarte populară

Exercițiul 1 – Rularea unui Model Local

- a. Instalați Ollama și descărcați un model local de tip Qwen3.
Porniți modelul în mod interactiv și inițiați o conversație simplă.
Verificați că modelul poate primi un prompt și genera un răspuns. Testați comportamentul acestuia folosind cel puțin trei întrebări diferite.
- b. Configurați Ollama astfel încât modelul să poată fi accesat prin intermediul API-ului local expus pe mașina voastră.
Verificați că endpoint-ul local răspunde corect la cereri și returnează răspunsuri generate de model.
La finalul exercițiului trebuie să puteți confirma că modelul rulează local și poate fi accesat programatic dintr-o aplicație Python.

Exercițiul 2 – Primul Agent Conversațional

Accesați fișierul config.py și completați valorile necesare pentru conectarea la modelul local.

Configurația trebuie să conțină:

- numele modelului utilizat;
- system prompt-ul agentului.

Hint: Modelul și promptul nu trebuie hardcodate în restul aplicației.

b. Accesați fișierul conversation_context.py și implementați metodele necesare pentru:

- adăugarea unui mesaj;
- generarea system prompt-ului
- returnarea istoricului conversației.

Hint: Toate mesaje trebuie păstrate într-o listă și vor fi transmise ulterior modelului.

c. Definiți un system prompt care stabilește comportamentul și rolul agentului.

Definiți personalitatea și tiparele modelului bazat pe atribuiri făcute în laborator.

După configurarea prompt-ului, verificați că răspunsurile modelului reflectă rolul ales.

Hint: System prompt-ul trebuie încărcat o singură dată la inițializarea unei conversații noi.

Exercițiul 3 – Primul Tool al Agentului

Utilizând scheletul de cod primit, completați implementarea funcției `lucky_number()` din fișierul `tools/lucky_number_tool.py`. Funcția trebuie să calculeze un număr norocos pe baza datei curente și a datei de naștere primite ca argument, conform regulii descrise în comentariile existente.

După implementarea funcției, accesați fișierul `tools/tools.py`, importați tool-ul creat și înregistrați-l în lista globală de tool-uri a aplicației.

La finalul exercițiului, porniți agentul și verificați că acesta poate utiliza tool-ul atunci când utilizatorul solicită generarea unui număr norocos.

Project Assignments

- | | |
|---------------------------|------------------------------|
| 1 - Python Mentor | 13 - Math Tutor |
| 2 - DevOps Engineer | 14 - Fitness Coach |
| 3 - Linux Wizard | 15 - Travel Consultant |
| 4 - Cybersecurity Analyst | 16 - Financial Advisor |
| 5 - Data Scientist | 17 - Game Master |
| 6 - AI Researcher | 18 - Dungeon Master |
| 7 - Software Architect | 19 - Productivity Coach |
| 8 - Code Reviewer | 20 - Life Coach |
| 9 - Technical Interviewer | 21 - Movie Critic |
| 10 - Product Manager | 22 - Book Recommender |
| 11 - Startup Founder | 23 - Chef & Recipe Assistant |
| 12 - College Professor | |

DIGITAL STACK

Knowledge. Performance. Growth.